Smart Housing Scout

Rent with evidence, not claims.

ASSIGNMENT 9: SOFTWARE PROJECT

A web app for students who rent in Pittsburgh without seeing the home first. It ranks homes by real door-to-campus commute time, true monthly cost, and verified bathroom count, instead of what the listing says.

Code: **github.com/srishti6singh/smart-housing-scout**

---

1. Functional decomposition

The digital experience: an international student choosing an apartment in Pittsburgh, near Carnegie Mellon, before arriving in the city.

User objective

The users are college students from overseas who need to find accommodation before they arrive. They want to see what a place really looks like, and to know three things a listing may not tell them honestly: how long it really takes to get from the door to campus, not just how far it is (the Commute Truth-Teller); whether “2 baths” means two full bathrooms or one bathroom and a toilet-only washroom; and what the home will really cost each month with utilities. This matters because they need actual numbers to plan: a student with a $700 monthly budget and a $500 rent needs to know that utilities will stay under $200.

Primary functions and system classification

#	FUNCTION	DONE BY	FRONT OR BACK END	WHERE IT MEETS THE REAL WORLD
1	Choose the campus and the time class starts ("Arrive by")	User	Front end	The student's real class schedule
2	Search by neighborhood and set filters: budget, beds, baths, max commute, utilities	User	Front end	Keyboard, mouse, or touchscreen
3	Turn "8:45" into the next weekday at 8:45 Pittsburgh time	Program	Back end	The real calendar and the Eastern time zone

#	FUNCTION	DONE BY	FRONT OR BACK END	WHERE IT MEETS THE REAL WORLD
4	Get a transit route from each home to the campus building	Program	Back end (server)	Google Routes API, which uses real bus schedules. Sample model when no key is set
5	Split the route into walk, wait, ride, transfer, final walk, and early arrival; compute total minutes	Program	Back end (logic)	Real stop locations and departure times
6	Estimate true monthly cost: rent plus electricity, gas, water, internet, insurance	Program	Back end (logic, runs in browser)	Square footage, heating type, season
7	Compare listed baths with verified full and half baths; raise flags	Program	Back end (logic)	Tenant photos and floor plans
8	Show cards, map pins, Journey Bars, photos, tables, and plain-language findings	Program	Front end	The screen; OpenStreetMap tiles
9	Save homes and pick 2 to 4 to compare	User	Front end	Stored in the browser (localStorage)
10	Decide which home to rent and contact the landlord	User	Outside the software	A real lease and a real move

Note: the commute logic runs on a small Node server so the Google API key never reaches the browser. The cost, bathroom, and comparison rules run in the browser, but they are still back-end logic: they apply rules, while the page only displays results.

2A. Project description and architecture

Project description

Smart Housing Scout is a commute truth teller and lease time machine. It lets you know exactly what your house is like from overseas and if it truly fits your budget. It shows your true daily commute to campus, based on bus schedules, along with real stats and verified reviews from past renters.

Target user

The users are international college students moving to Pittsburgh who need to secure accommodation before they arrive.

Requirements

The app must rank homes by real door to campus commute time rather than distance. It needs to break the trip down into a visual Journey Bar, calculate the true monthly cost including utilities, flag

mismatches in bathroom counts, let users compare options side by side, and work offline using sample data when no Google API key is set, and show a clear error message if the server is down.

#	REQUIREMENT	MET?
R1	Rank homes by door-to-campus commute time by default, with distance as secondary text	**Yes**
R2	Break each trip into walk, wait, ride, transfer, final walk, and early arrival, and draw it as a proportional Journey Bar	**Yes**
R3	Say in plain words when a closer home is slower ("Closer than Penn Ave, but 21 min slower")	**Yes**
R4	Show the true monthly cost with a toggle per utility and a season choice; label every estimate	**Yes**
R5	Flag a mismatch between listed and verified baths	**Yes**
R6	Show unstaged tenant photos, a before/after slider, and verified tenant reviews	**Yes**
R7	Compare 2 to 4 homes and highlight the best value in each row	**Yes**
R8	Keep the Google API key on the server; work fully offline with sample commutes when no key is set	**Yes, sample mode tested. Google mode not tested (no key)**
R9	Loading, empty, and error states; keyboard focus; works on a phone	**Yes**

Inputs and outputs

Inputs include the target campus, a specific arrive by time, search text, and filters for budget, beds, and baths. Outputs provide the step by step commute breakdown, estimated true monthly utility cost, verified tenant ratings, and plain language mismatch flags.

INPUTS	OUTPUTS
Campus (CMU, Pitt, Duquesne)	Commute: total minutes, each segment, bus lines, transfers, leave and arrive times
Arrive-by time (7:00 to 11:00 AM)	
Search text	True monthly cost and each utility estimate ($)
Filters: max budget, min beds, min verified full baths, max commute, utilities included	Bath check: listed vs verified, mismatch flag
	Lease flexibility score (0 to 4)
Sort order	Tenant rating (1 to 5)
Per-home: season, "included in rent" toggles	Flags and plain-language findings

Fixed values

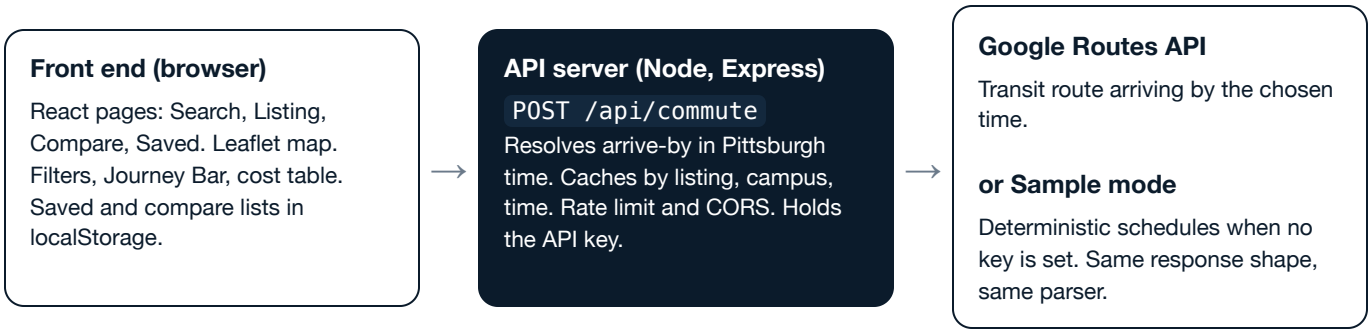
ITEM	VALUE
Campus buildings (where trips end)	CMU Gates Hillman Center; Pitt Cathedral of Learning; Duquesne Old Main
Time zone	America/New_York, computed on the server

ITEM	VALUE
Electricity ($ per sq ft per month)	Winter 0.085, summer 0.12, other months 0.07; electric heat adds 0.20 in winter
Gas ($ per sq ft per month)	Winter 0.17, summer 0.025, other months 0.06
Water and sewer; internet; insurance	$22 + $16 per bedroom; $65; $15
Building efficiency multiplier	Efficient 0.85, average 1.0, drafty 1.4 (on heating and cooling)
"Meaningful" commute difference	3 minutes or more

How it works (logic and math)

If you select CMU, the app finds the actual bus going to CMU from a specific stop rather than a random one, aligning with transit APIs for accurate schedules (sample schedules until a Google API key is added). For the commute math, the total time equals your arrive by time minus your departure time, meaning early arrival is accurately counted as commute time. True cost is calculated by adding rent and estimated utilities based on square footage, season, and heating type. The Bathroom Detective compares the listed count against verified data where a half bath equals 0.5. It raises a red flag if the numbers do not match.

Architecture



Shared code (`shared/`) is used by both sides: types, sample listings, the route parser, the cost model, and the comparison rules.

Success metrics

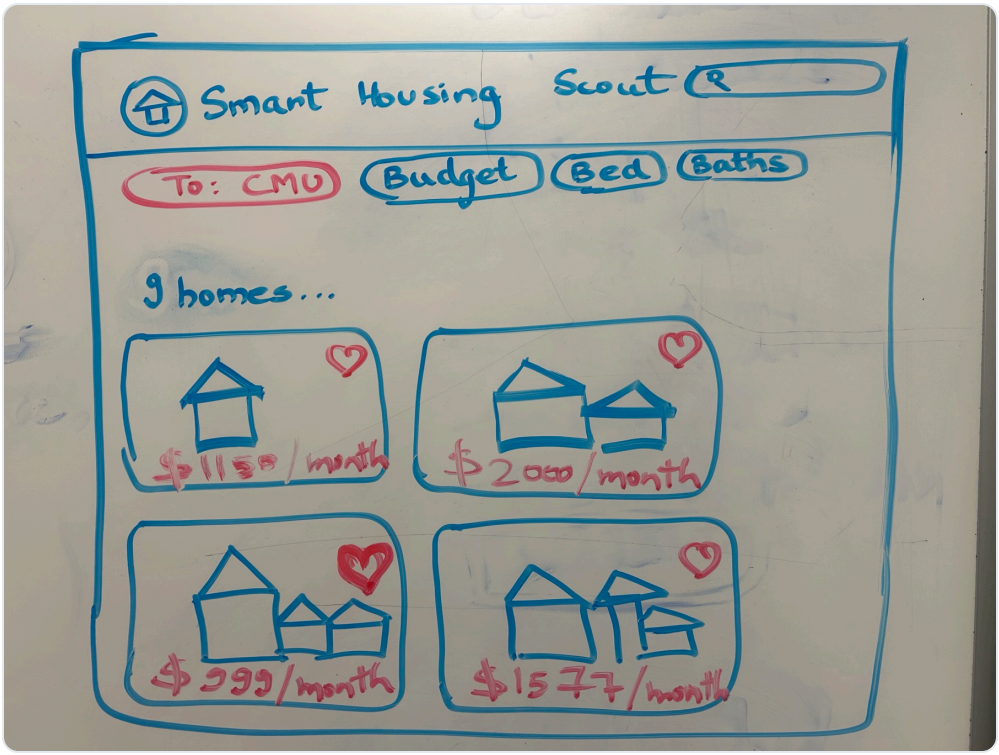
Success looks like students actually using the app to finalize houses without depending on Zillow, Google Maps, or video calling. Technically, success means all five test traps trigger correctly in the frontend development, commute segments perfectly add up to the total time, and the cost table perfectly matches manual calculations.

Testing plan

The testing plan relies on five built in traps to catch edge cases: a home that is physically close but has a slow commute, a farther home with a fast direct bus, a listing claiming 2 baths that is verified as 1.5, a cheap rent with hidden high utilities, and a home with great reviews but the longest commute.

TRAP	HOME	EXPECTED RESULT (CMU, ARRIVE 8:45 AM)	RESULT
Closest home, slow commute	Semple St, 0.8 mi	39 min: 75 bus then 61C, arrives 14 min early. Not the fastest	**Pass**
Farther home, fast direct bus	Penn Ave, 2.3 mi	18 min on the 71D, no transfer. Fastest of 12	**Pass**
Listed 2 baths, really 1.5	Ellsworth Ave	Red mismatch flag: 1 full + 1 half	**Pass**
Cheap rent, high utilities	Greenfield Ave	Rent $950, true cost $1,405 (average), $1,560 in winter	**Pass**
Great reviews, long commute	Beechwood Blvd	Rating 4.8 of 5, commute 43 min (slowest)	**Pass**

Interface sketch



Hand-drawn sketch of the search page, made before building.

2B. Working demonstration code

The full code is on GitHub: github.com/srishti6singh/smart-housing-scout. To run it: `npm install`, then `npm run dev`, then open localhost:5173 (details in the README).

1. Initial prompt

Submitted to Claude Code in one message. The full text is below, unchanged.

> You are a senior product designer and full-stack engineer. Build "Smart Housing Scout", a rental search web app for students who rent sight-unseen. It should have the structure of Zillow (map and list side by side, filters, saved homes) and the craft of Airbnb (clean, calm, image-led,

every detail finished). Its difference: every claim in a listing is checked and shown as evidence.

PRODUCT
- Name: Smart Housing Scout. Tagline: "Rent with evidence, not claims."
- Users: students and international students relocating to a city they have never visited. Default city: Pittsburgh. Default campus: Carnegie Mellon University. The campus selector includes CMU, University of Pittsburgh, and Duquesne University.
- Core idea: listings say "5 minute walk to campus", "2 bath", "utilities included", and show staged photos. Scout replaces each claim with something verified.

THREE SIGNATURE FEATURES
1. Commute Truth-Teller (the MVP; build it first and best)
 - Compares homes by true door-to-campus commute time, not distance.
 - For each listing, get a transit route from the listing's coordinates to the selected campus building, arriving by a chosen time (default 8:45 AM on the next weekday).
 - Break the trip into four parts: walk to stop, wait, ride, walk to building. Wait = transit departure time minus (trip start + walk-to-stop time). Show line names, number of transfers, and total minutes.
 - Signature visual: a proportional "Journey Bar", four colored segments scaled to minutes (walk teal, wait amber, ride blue, final walk light teal), with minutes labeled.
 - Default sort is "Fastest commute". Show distance as secondary text. Where the closest home is not the fastest, say so in plain words, for example "Closer than Maple St, but 11 min slower".
2. Lease Time Machine (shows what a listing hides), on the detail page as three tabs:
 - Real photos: unstaged, multi-angle room photos uploaded by past tenants, each with room label, angle, and date. Include a before/after slider that compares the listing photo with the unstaged photo of the same room. State clearly that photos are unstaged and not AI-edited.
 - True monthly cost: rent plus estimated electricity, gas, water and sewer, internet, and renters insurance, as a stacked bar and a table. Headline: "Listed $1,450. True monthly cost $1,687." A toggle for each utility lets the user mark it "included in rent". Estimates come from sample factors per square foot and season, labeled "estimate".
 - Past tenants: overall rating plus category ratings (landlord responsiveness, maintenance, noise, winter heating), and two or three short reviews with tenancy dates and a "Verified tenant" badge.
3. Bathroom Detective
 - Compare the listed bath count with the verified breakdown (full and half baths). If they differ, show a red mismatch flag with a one line explanation.
Also include a Lease terms panel (length options, deposit, early termination, sublease allowed, notice period) and a short accordion of GENERAL tenant-rights information, labeled "general information, not legal advice". Do not invent specific laws or statute numbers.

SCREENS
1. Search (home): sticky top bar with logo, one rounded search field (neighborhood, address, or campus), an "Arrive by" time picker, and filter pills (budget, beds, baths, max commute minutes, utilities). Below it, a split view. Left: scrollable result cards. Right: a map with price pins. Hovering a card highlights its pin, and clicking a pin scrolls to its card. On mobile: a map and list toggle with a bottom sheet.
2. Listing card: image carousel with dots and a heart (save), then price, beds, baths, square feet, address, a commute chip ("31 min door to campus") with a mini Journey Bar, and small badges ("Baths verified", "Unstaged photos", "True cost +$237").
3. Listing detail: Airbnb-style gallery (one large image and four small, with "Show all photos"), a sticky right panel with the true monthly cost and Save, Compare, and Contact buttons, and sections for Commute Truth-Teller, Lease Time Machine, Bathroom Detective, Lease terms, and neighborhood.
4. Compare (2 to 4 homes): a pivot table with homes as columns and rows for true commute (Journey Bars), true monthly cost, baths (listed vs verified), lease flexibility, tenant rating, and flag count. Highlight the best value in each row.
5. Saved homes.

DESIGN SYSTEM: Zillow's structure, Airbnb's finish
- Light, airy listing surfaces: white (#FFFFFF) cards over mist (#F6F8FA). Deep midnight navy (#0B1B2B) for the header, nav, and the evidence panels. Evidence values (minutes, dollars, counts) use tabular numerals or a monospace face, so verified data looks different from listing claims.
- Colors: primary action Scout green #0E9F6E, warning and hidden cost amber #F5A524, mismatch red #E5484D, transit blue #2F6FED, hairline borders #E3E8EE. Do not copy Zillow's blue or Airbnb's red.
- Type: Inter for UI, tabular numerals for numbers, one display weight for headings. Scale 12 / 14 / 16 / 20 / 28 / 40, line height 1.4 to 1.5, line length under 70 characters.
- Layout: 8 pixel grid. 16 to 20 pixel radius on images and cards, 10 to 12 on inputs, fully rounded pills and buttons. Soft layered shadows only on hover and on floating elements. Generous whitespace. Photos are the hero of every card.
- Motion: small and purposeful only. Card lift on hover, heart pop on save, carousel slide, Journey Bar segments growing in once, skeleton loaders. Respect reduced-motion settings.
- States: loading skeletons, empty states with a clear next step, error states that say what happened and how to fix it. Visible keyboard focus. WCAG AA contrast. Fully responsive.
- Copy: plain and specific. No marketing filler. No lorem ipsum.

DATA
- Use clearly labeled sample data: 12 fictional listings across Shadyside, Squirrel Hill, Oakland, Bloomfield, Friendship, Point Breeze, and Greenfield, with realistic coordinates, rents ($900 to $2,400), unit photos, utilities, reviews, and lease terms. Use royalty-free placeholder images with stable IDs.
- Build in these test traps: one home that is closest but has a slow commute, one farther home with a fast direct bus, one listing that says 2 baths but is verified 1 full and 1 half, one cheap rent with high utilities, and one with excellent tenant reviews but a long commute.
- Define TypeScript types for Listing, CommuteResult (walkToStop, wait, ride, walkToBuilding, transfers, lines, totalMinutes), CostBreakdown, TenantReview, and BathroomCheck.

TECH
- React with TypeScript, Vite, Tailwind CSS, and React Router. Leaflet with OpenStreetMap tiles for the map, so no map key is needed.
- A small Node and Express backend that proxies Google's Routes API for transit: POST https://routes.googleapis.com/directions/v2:computeRoutes with travelMode TRANSIT and arrivalTime, using the headers X-Goog-Api-Key and X-Goog-FieldMask. Request only the fields needed (routes.legs.steps.transitDetails, step durations, start and end times, total duration). Keep the API key on the server in an environment variable, never in the front end. Provide a .env.example.
- Endpoint: POST /api/commute with origin, destination, and arriveBy. Cache results in memory by listing, campus, and time. Add a simple rate limit and CORS.
- Mock mode: if no API key is set, return deterministic sample commutes and show a small "Sample commute data" label, so the app works fully offline.
- Write unit tests for the function that turns a Routes API response into the four journey parts, using a fixture JSON.

OUT OF SCOPE
User accounts, payments, real listing scraping, and the Zillow API.

HOW TO WORK
Build in this order: (1) design tokens and layout shell, (2) sample data

and types, (3) search page with map and cards, (4) Commute Truth-Teller with backend and mock mode, (5) listing detail with the Lease Time Machine and Bathroom Detective, (6) compare page and saved homes, (7) polish and accessibility. Do not ask me questions. Make reasonable assumptions and list them at the end. Do not claim anything works until you have run it, and tell me exactly how to run it and where to add the Google API key.

REVIEW NOTES SENT WITH THE PROMPT (SUMMARY)

The same message included a review of the prompt. Claude applied each point:

- **Early arrival:** with "arrive by 8:45", the real loss is landing at 8:22. Added as a fifth segment; total = arrive-by – departure.
- **Transfers:** a trip with a transfer has two rides and two waits. The Journey Bar shows the trip as an ordered sequence.
- **Routes API field names:** check the docs. Claude confirmed that steps use `staticDuration` and that a route has no departure time.
- **Campus building** and **time zone** (America/New_York on the server) defined.
- Utility factors written out, photo pairs labeled illustrative, localStorage for saved homes, "best" defined per Compare row, tests beyond the parser, commit after each stage.

2. The finished interface

All screenshots: captured by Claude with headless Chrome from the running app, Nov 9, 2026, sample commute data.

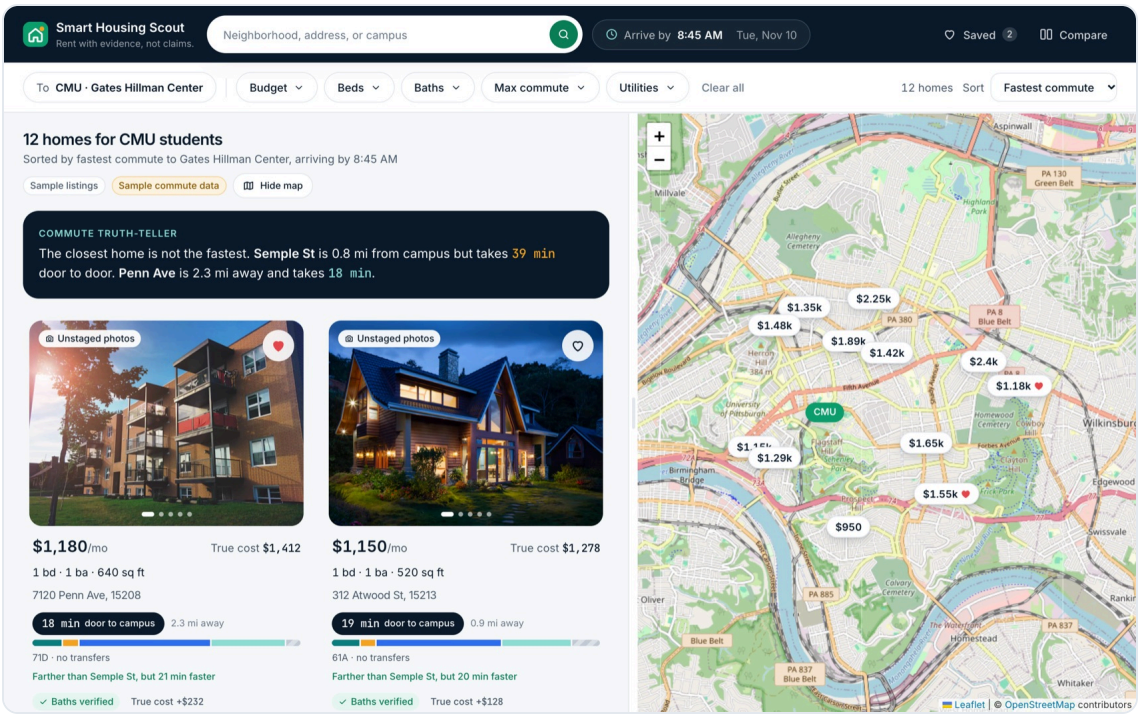


Figure 1. Search page. Cards sorted by fastest commute, map with price pins, and the finding "The closest home is not the fastest". The bar between list and map resizes the map; "Hide map" removes it.

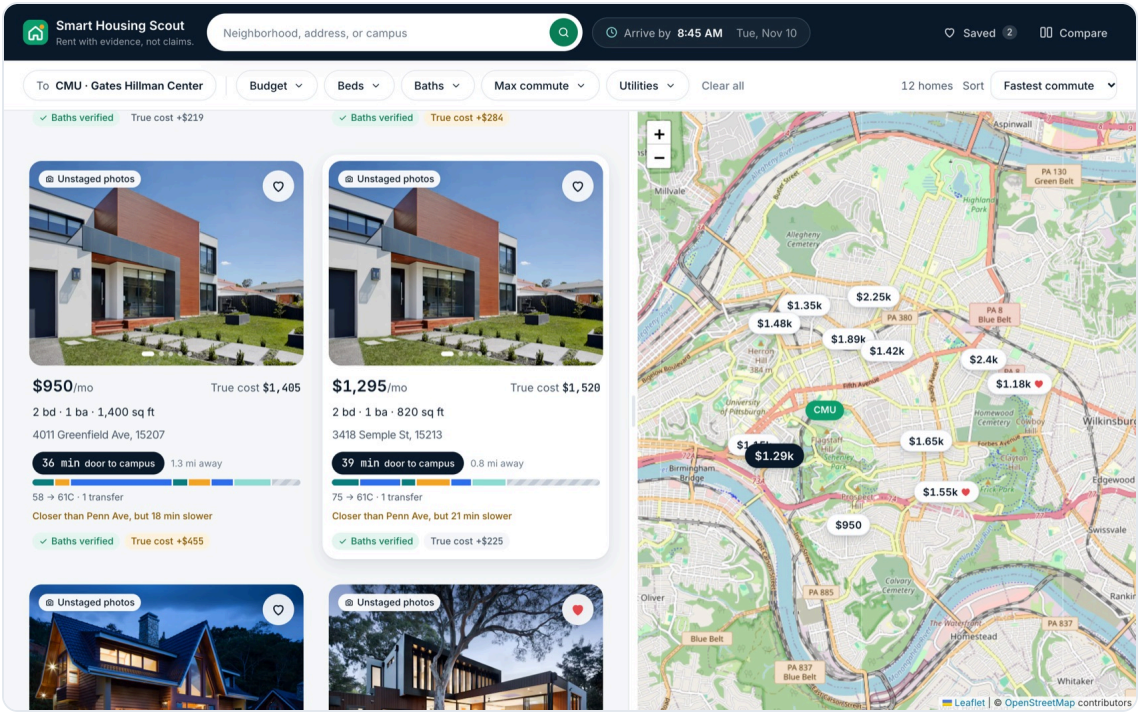


Figure 2. Hovering the Semple St card highlights its pin on the map ($1.29k, dark). Each card shows the mini Journey Bar and a finding such as "Closer than Penn Ave, but 21 min slower".

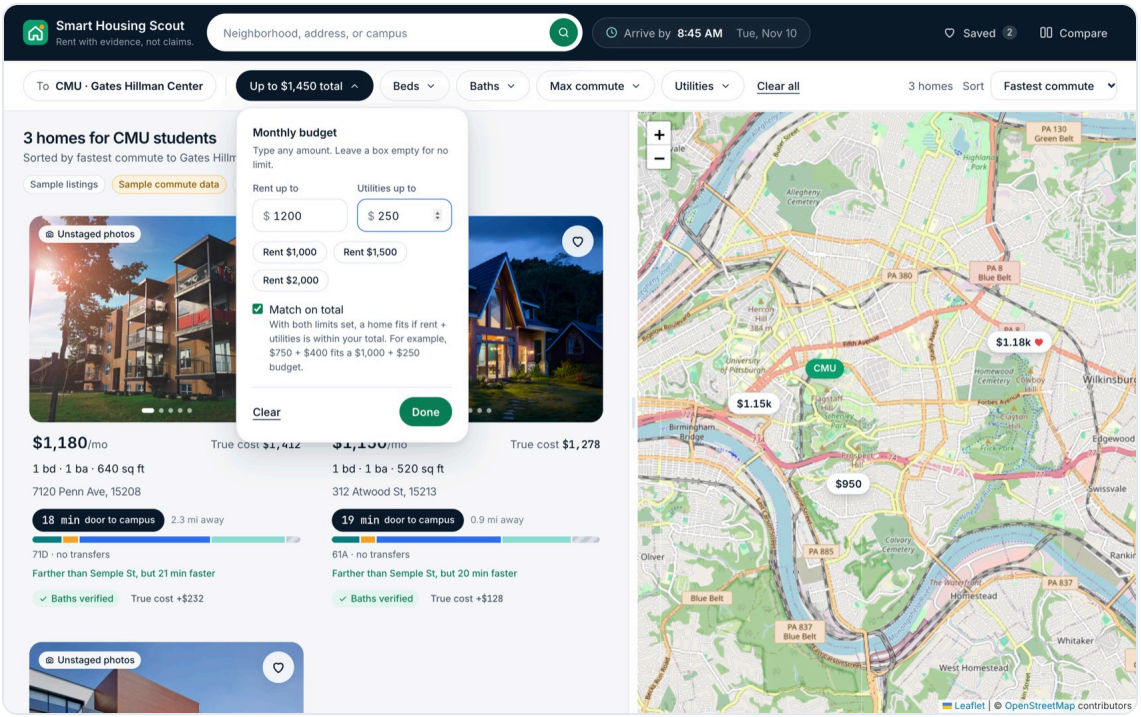


Figure 3. Budget filter with typed limits: rent $1,200 and utilities $250. With "Match on total", 3 homes fit within $1,450 in total. Every filter has its own Clear, and "Clear all" resets them.

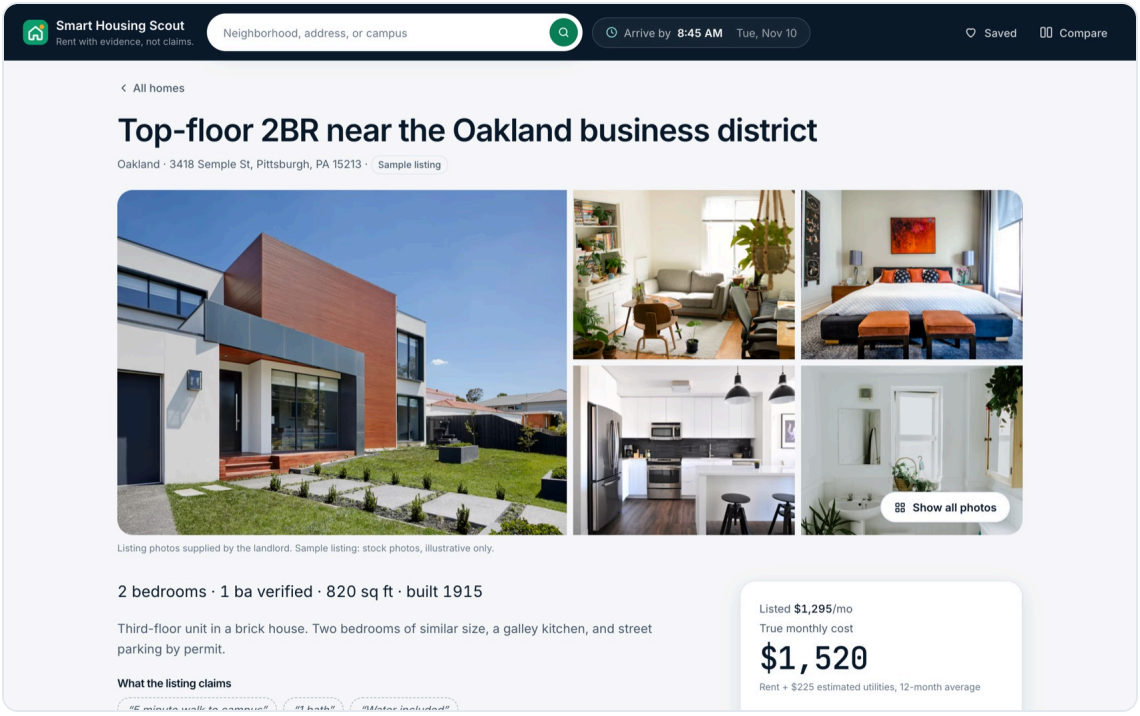


Figure 4. Listing detail: gallery (one large, four small), the listing's own claims, Scout's flag, and the sticky cost panel.

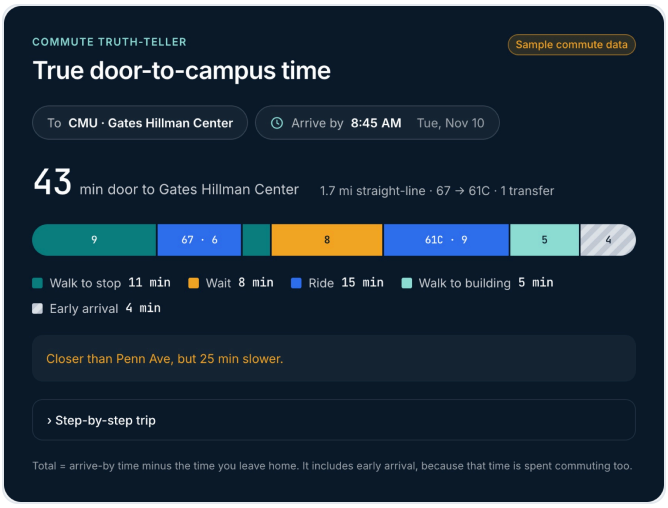


Figure 6. The best-reviewed home (Beechwood Blvd) has the longest commute: 43 min, infrequent 67 bus plus a transfer.

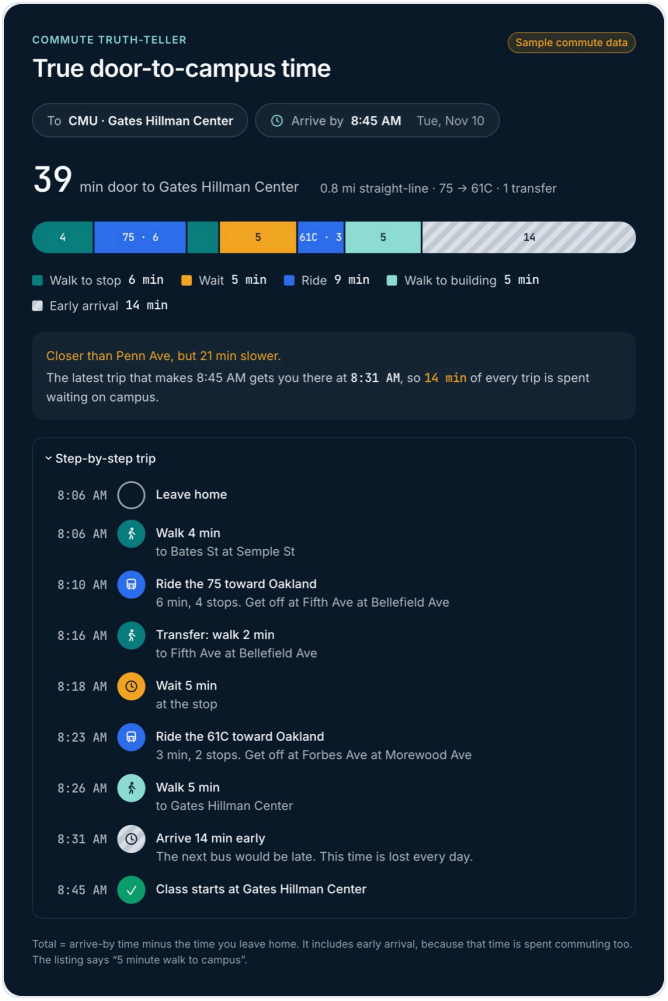


Figure 5. Commute Truth-Teller for the closest home. The listing says "5 minute walk to campus"; the real trip is 39 min with a transfer and 14 min of early arrival.

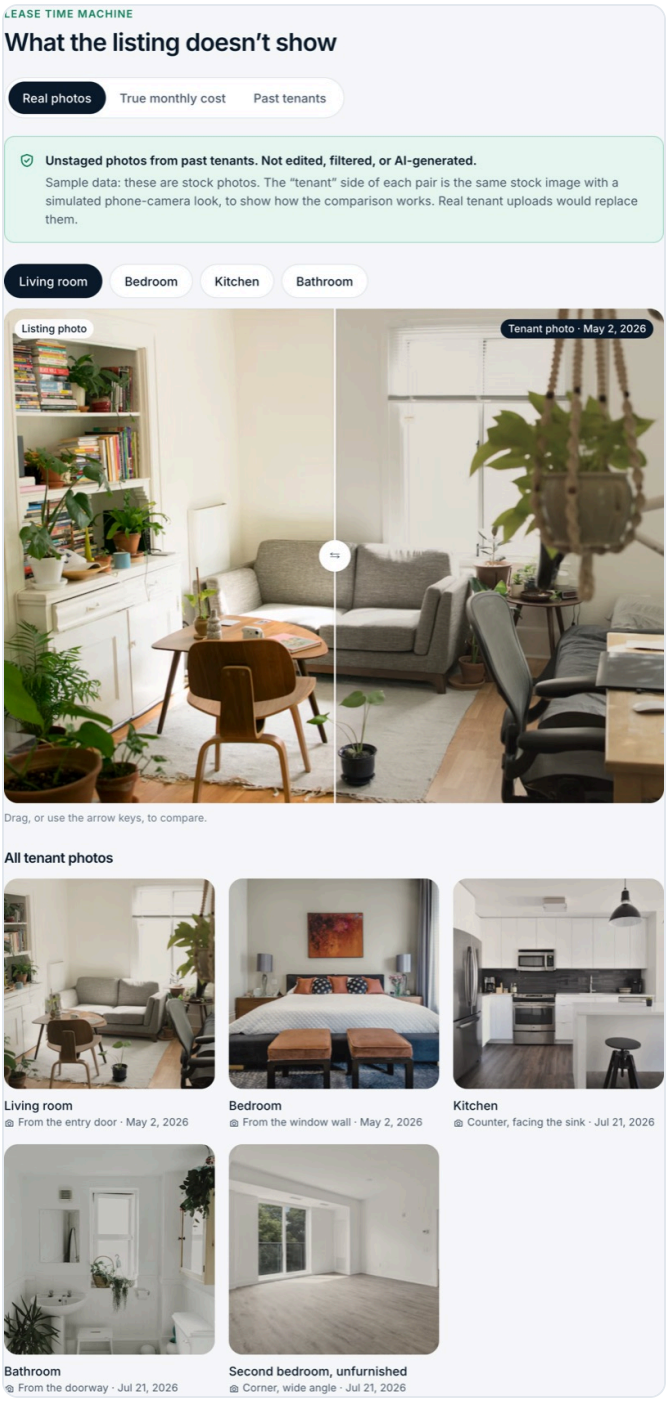


Figure 7. Lease Time Machine, Real photos tab: before/after slider and dated tenant photos, with a note that these are illustrative stock pairs.

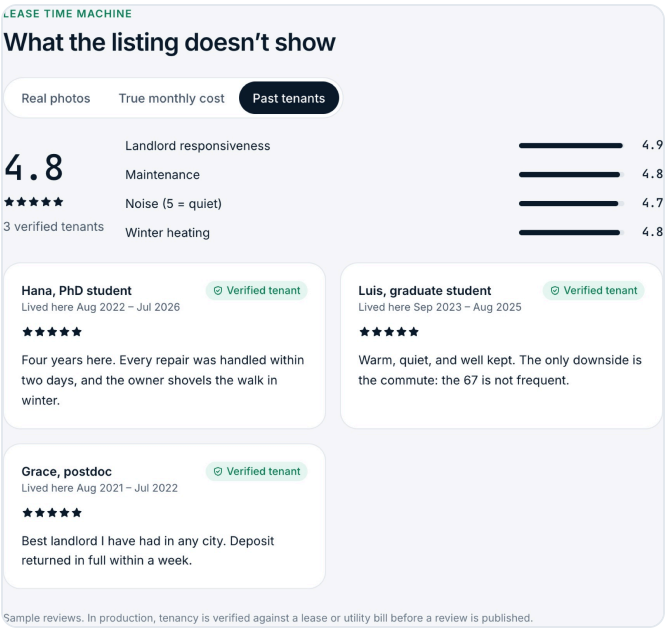


Figure 9. Past tenants tab: category ratings and reviews with a Verified tenant badge and tenancy dates.

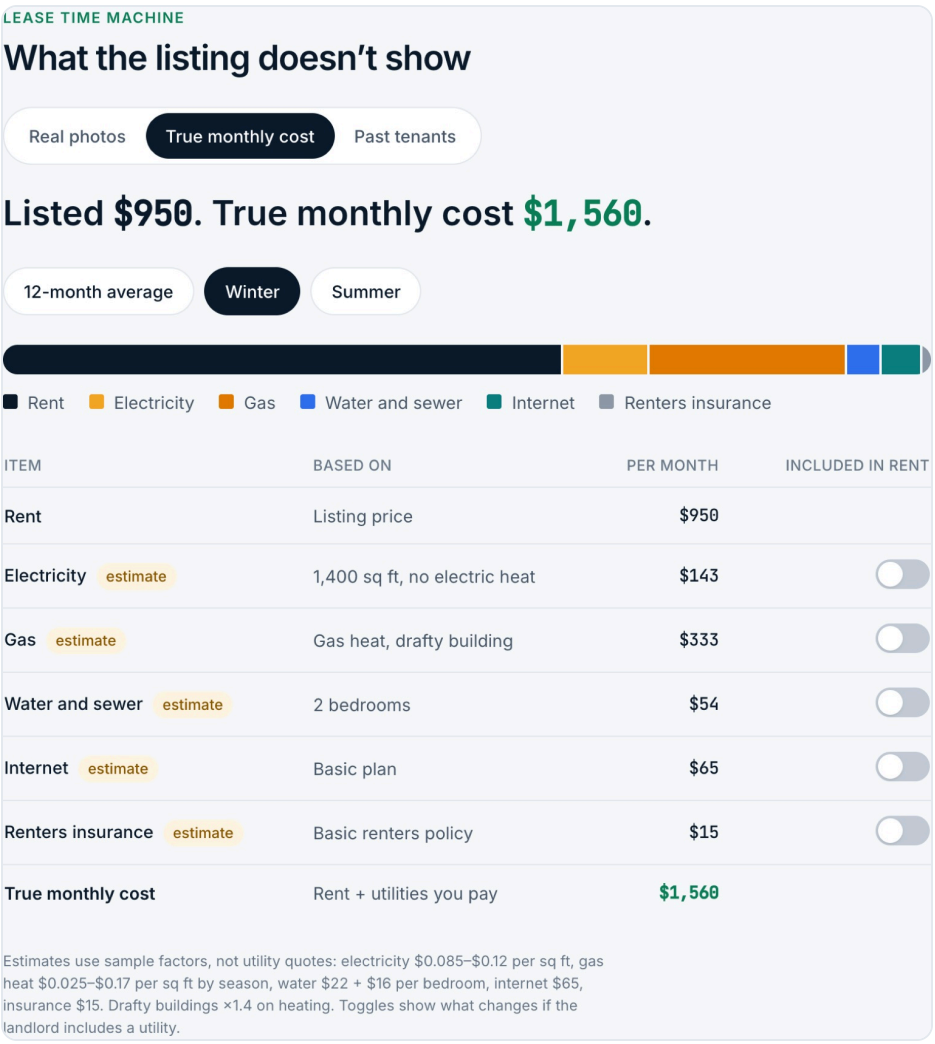


Figure 8. True monthly cost for the cheapest listing, winter: "Listed $950. True monthly cost $1,560." Gas alone is $333 in a drafty house.

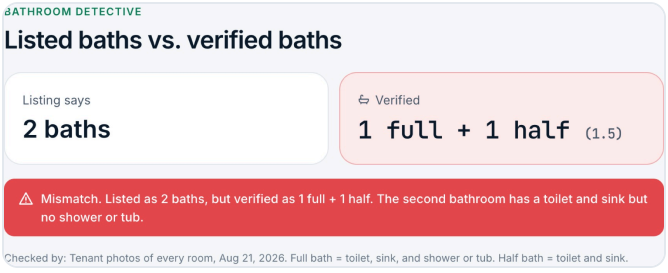


Figure 10. Bathroom Detective: listed 2 baths, verified 1 full + 1 half, red mismatch flag.

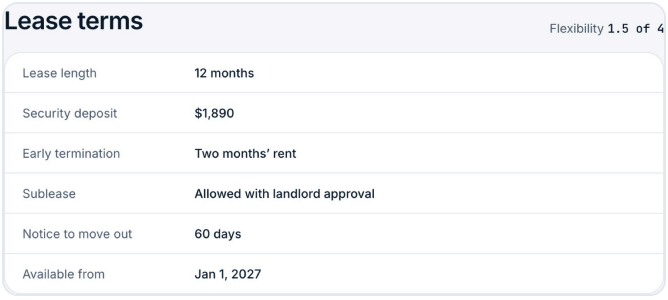


Figure 11. Lease terms with a flexibility score.

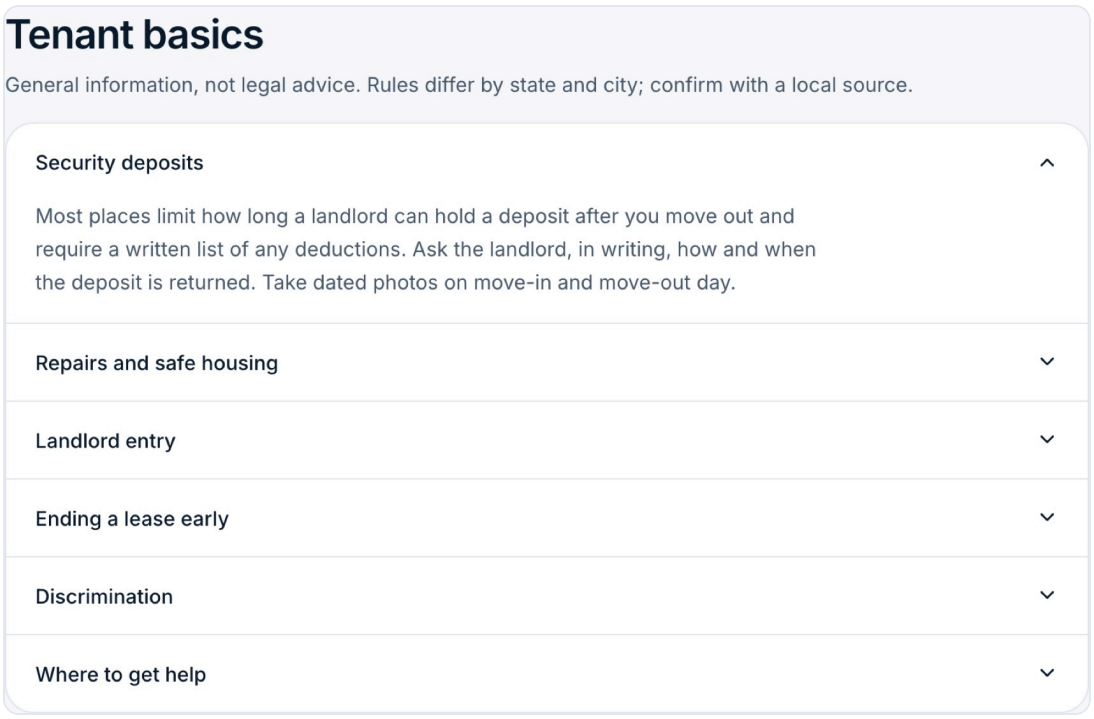


Figure 12. Tenant basics, labeled "general information, not legal advice". No statute numbers.

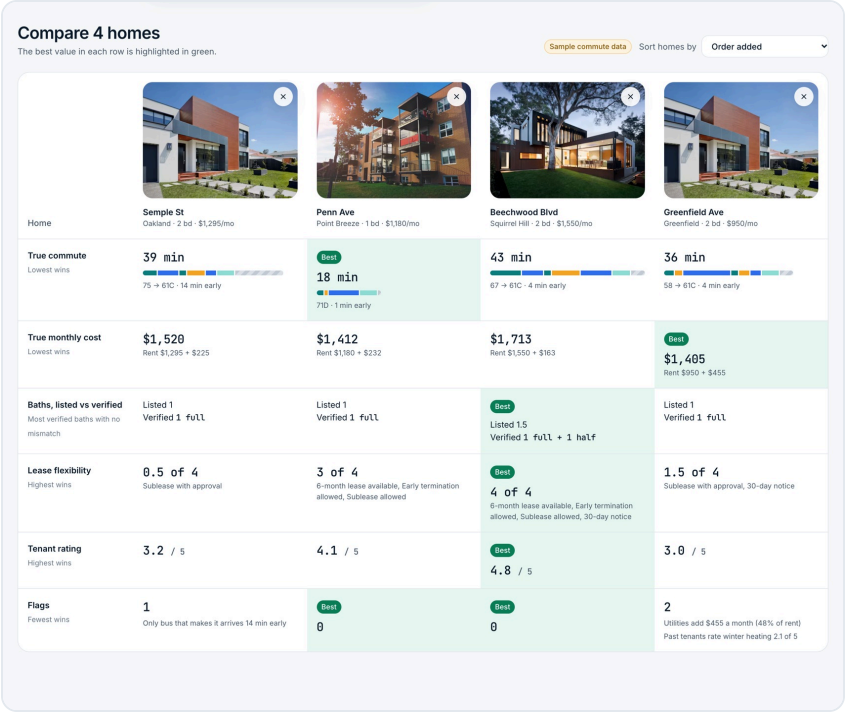


Figure 13. Compare 4 homes. The "Sort homes by" menu reorders them by commute, utilities, true cost, rent, rating, or flags. The best value in each row is green: lowest commute and cost, most verified baths with no mismatch, highest flexibility and rating, fewest flags.

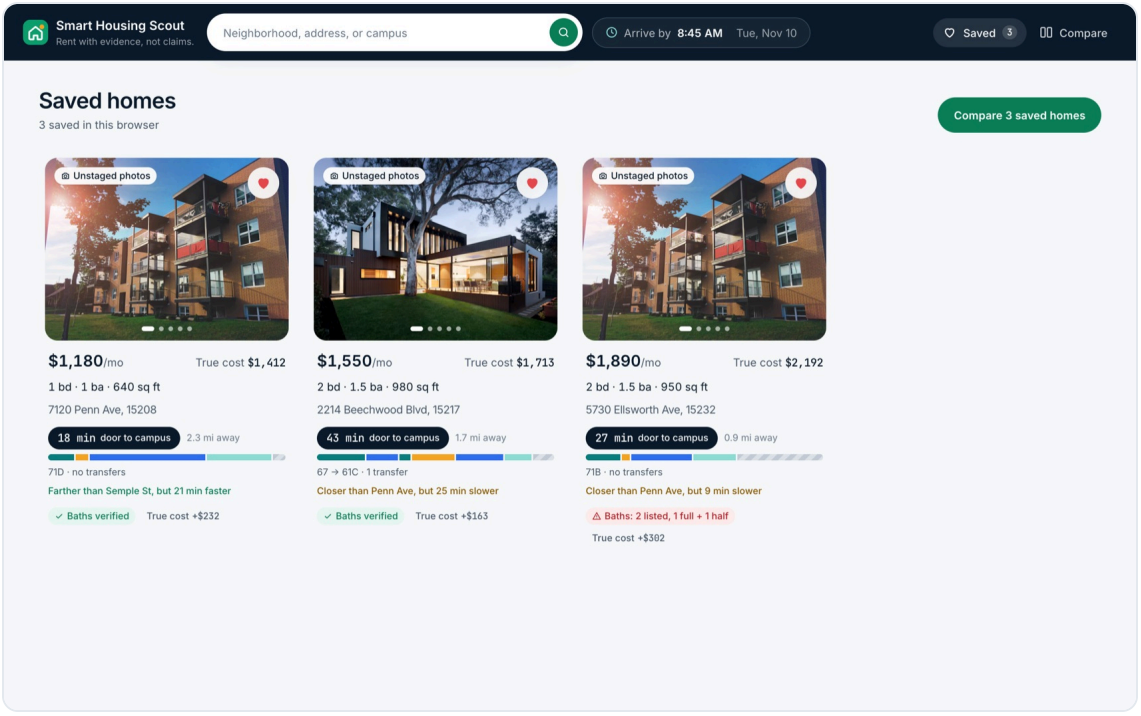


Figure 14. Saved homes, stored in the browser.

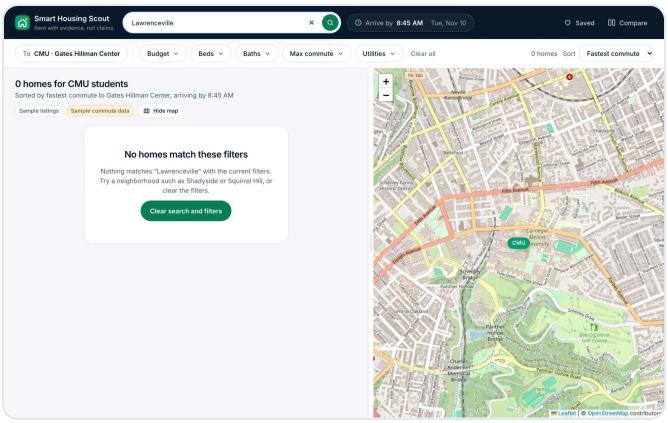


Figure 15. Empty state with a clear next step.

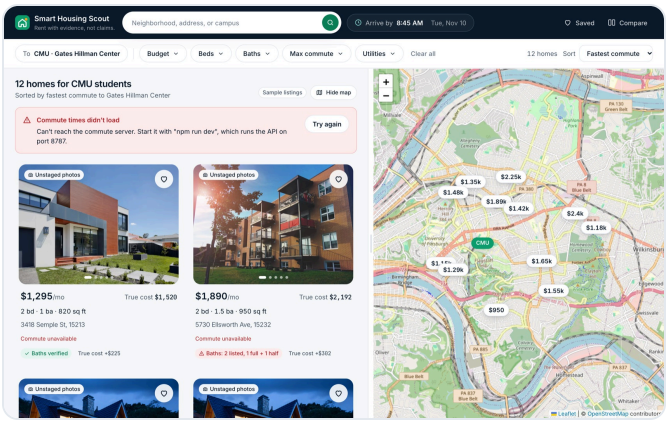


Figure 16. Error state when the commute server is off: what happened, how to fix it, and a Try again button.

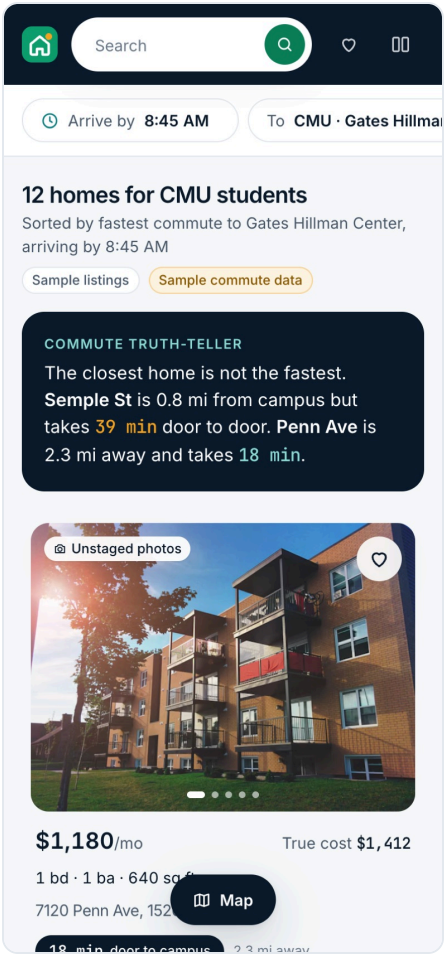


Figure 17. Phone: list view.

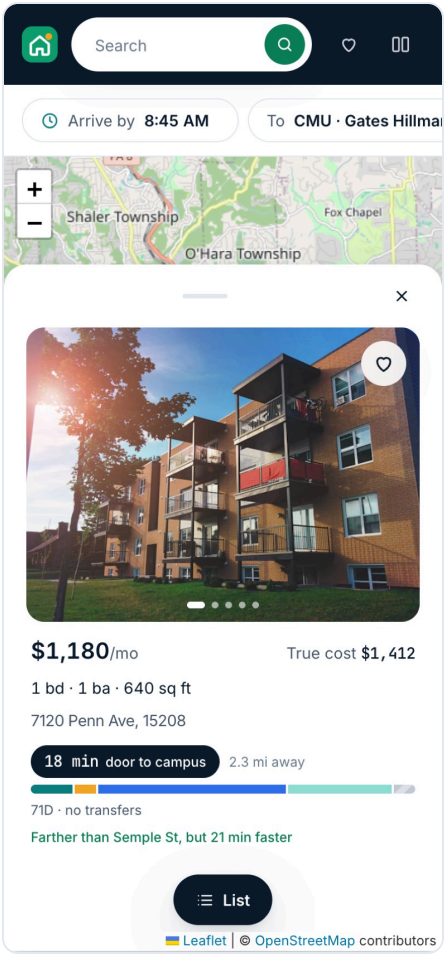


Figure 18. Phone: map view; tapping a pin opens a bottom sheet.

3. Testing verification

Automated tests, run with `npm test` on Nov 9, 2026. All 35 pass. The fixture is a Google Routes API response with one transfer.

AREA	WHAT THE TESTS CHECK	TESTS	RESULT
Route parser	A Google Routes response with one transfer (fixture) becomes walk 5, wait 5, ride 13, transfer walk 2, final walk 4, early arrival 10 = 39 min. Lines 61B and 71A, 1 transfer. Edge cases: missing duration, walk-only route, no route, odd seconds still add up	11	**All pass**
True cost	Hand calculation for a 1,000 sq ft, 2-bedroom, gas-heated home: $92 + $85 + $54 + $65 + $15 = $311 extra. Included utilities, toggles, winter vs summer, drafty buildings	6	**All pass**
Findings	"Closer than Penn Ave, but 21 min slower" and "Farther than Semple St, but 21 min faster"; differences under 3 min ignored; 2 listed vs 1 full + 1 half flagged; 1.5 listed vs 1 full + 1 half accepted; lease flexibility score	8	**All pass**
Sample traps	Closest home is not the fastest; a farther direct bus wins; the best-reviewed home has the longest commute; results repeat exactly	3	**All pass**

AREA	WHAT THE TESTS CHECK	TESTS	RESULT
Time zone	"8:45" becomes the next weekday at 8:45 in Pittsburgh: Monday afternoon to Tuesday, Friday to Monday, late Thursday night stays Thursday, summer daylight time, bad input rejected	7	**All pass**
Total (`npm test` , Nov 9, 2026)		**35**	**35 of 35 pass**

Manual checks in a browser: toggling "Internet included" lowered the Ellsworth Ave total from $2,192 to $2,127 (–$65); hovering a card highlighted its pin; the API rejected `"8.45"` as an arrive-by time with a clear message; a request from another website got no CORS permission.

2C. Code modification

The change

After using the app myself, I asked for four changes. The main one was the budget filter. Before, I could only pick a preset amount such as “Under $1,500”. Now I can type my own limit for rent and for utilities, and turn on “Match on total” (Figure 3).

The other three: the map can be resized by dragging the bar next to it, or hidden; every filter has its own Clear button, utilities has Select all, and there is a Clear all; and the Compare page can sort homes by shortest commute, lowest utilities, or lowest total cost.

Why

While browsing, I could not enter my own numbers. If my budget is $1,000 for rent and at most $250 for utilities, a house with $750 rent and $400 utilities should still show up, because the total fits. That is the same budget question as my user objective. The map always took about 30% of the right side of the home page, which did not make sense when I only wanted to look at the homes. And when comparing, I wanted to line homes up by commute or cost, the way you sort a list.

How

Claude wrote the code, and I tested it. The budget filter now stores three things: a rent limit, a utilities limit, and whether to match on the total. A small function, `budgetMatch` , checks a home: when both limits are set and “Match on total” is on, rent plus utilities must be within the two limits added together; otherwise each limit is checked on its own. Eight new unit tests check this, including my $750 + $400 example. In the browser, rent $1,200 and utilities $250 showed 3 homes with “Match on total” on and 2 with it off.

Testing the change also found a bug: the filter panel closed whenever the window resized. On a phone, tapping a number box opens the keyboard, which resizes the window, so the panel would close before you could type. It now moves into place instead of closing.

2D. Summary reflection

Systems approach. Before any code was written, I wrote one detailed prompt: who the users are, the three features, five test traps with the result I expected from each, the order to build in, an offline sample mode, and a rule that nothing counts as working until it has been run. Before building, I had the prompt reviewed, and the review found the biggest gap: “arrive by 8:45” hides the time lost by arriving early. That became a fifth part of the trip, and the total commute is now the arrive-by time minus the time you leave home. The five traps worked as my test plan, because each one has a result you can check.

Plan compared with the finished project. The main logic went as planned, but several things did not. My laptop did not have Node.js installed. The traps did not appear on their own; the sample bus schedules had to be adjusted until the closest home really was slow and the far one really was fastest. Google’s Routes API does not give a trip start time, so it had to be worked out from the trip length. And once I used the app myself, I asked for four changes that were not in my plan: typing my own budget, resizing the map, clearing filters, and sorting on the Compare page. The real Google transit data is still untested, because I do not have an API key yet.

Working with the AI. The AI was most helpful at turning a detailed prompt into a full working app quickly, with automated tests. It still needed checking. All 35 tests passed while the app showed a house with palm trees as a Pittsburgh listing, a confusing comparison between homes, and a phone layout where the search box was cut off; screenshots caught those. My own browsing found the changes that mattered most to a student, like typing my own budget. I stayed the director: I chose the features, the test traps, and the changes after testing.

Next time. I would write the exact expected number for each trap in the prompt, sketch the phone layout as well as the desktop one, get a Google API key early so real data can be tested, and try the app as a user sooner, because my own testing found the most useful improvements.

---

3. Group review

In our group discussion, each of us presented our project and the others tried it. Gautam built a poker game with different game modes, such as a biggest-hand game. His interface was great: very clear, with strong colors and contrast, and very interactive. It felt like a full website, not something made just for an assignment, so hats off to him.

Arzu made a bus time checker that tells you when to take the bus. Her interface was very cute, and it removed the boring friction from a daily routine. Tanya made an internship tracker that was simple yet interactive. Max built the backend for the common space project, and I loved how detailed it was, with the punk wall, the desk LED, and the ring countdown.

Tvisha’s project was the one I found most helpful for me personally. Her tracker keeps track of what is left in the fridge and helps plan what to make each day. As a student at Carnegie Mellon, deciding what to cook every day without knowing what is left in the fridge is a real struggle, and her tracker

handles all of it in a simple, organized way. Right now nothing like it exists unless you ask Claude to manage it for you.